

إليه الكلمة الأساسية التي نستخدمها عشان نعرّف class في Python؟

object **أ**

class **ب**

def **ج**

type **د**

إليه وظيفة الـ `__init__` في الـ class؟

بتحذف الـ object لما ينتهي **أ**

بتشغل تلقائي لما بنعمل object جديد ويتحدد الـ attributes **ب**

بترجع قيمة الـ class **ج**

بتطبع اسم الـ class **د**

إليه التي يحصل لو حاولت تعدي على attribute معرّف بـ double underscore (`__`)؟

```
"kw">class Foo:
    "kw">def __init__("kw">self):
        "kw">self.__secret = 42

f = Foo()
"kw">print(f.__secret)
```

هتطبع 42 **أ**

هتارجع None **ب**

هتحصل AttributeError لأن الـ name mangling بيخفيها **ج**

هتطبع 0 **د**

الكود ده هيطبع إيه؟

```
"kw">class Animal:
    "kw">def __init__("kw">self, name):
        "kw">self.name = name
    "kw">def speak("kw">self):
        "kw">return f">self.name} makes a sound"

"kw">class Dog(Animal):
    "kw">def __init__("kw">self, name, breed):
        "kw">super().__init__(name)
        "kw">self.breed = breed
    "kw">def speak("kw">self):
        "kw">return f">self.name} says Woof!"

d = Dog("Rex", "Labrador")
"kw">print("kw">isinstance(d, Animal))
"kw">print(d.speak())
```

False Rex makes a sound **أ**

!True Rex says Woof **ب**

True Rex makes a sound **ج**

Error: can't inherit from Animal **د**

إيه المفهوم اللي بيخلي الكود ده يشتغل؟

```
shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]
for s in shapes:
    "kw">print(s.area())
```

Encapsulation — عشان الـ data متخبية **أ**

Abstraction — عشان في ABC **ب**

Polymorphism — نفس الـ method name بس سلوك مختلف في كل class **ج**

Constructor Overloading **د**

في الكود دم، إيه الفرق بين bank و balance؟

```
"kw">class BankAccount:
    bank = "Python Bank"    # (A)
    "kw">def __init__("kw">self, owner, balance=0):
        "kw">self.owner    = owner
        "kw">self.balance = balance    # (B)
```

أ (A) و (B) كلهم زي بعض، مفيش فرق

ب (A) class attribute مشترك بين كل الـ (B) instance attribute، objects، خاص بكل object

ج (A) class attribute، (B) instance attribute

د (A) بيتحذف بعد الـ __init__

إيه اللي هيحصل لو حاولت تعمل instance من Abstract class مباشرة؟

```
"kw">from abc "kw">import ABC, abstractmethod

"kw">class Vehicle(ABC):
    @abstractmethod
    "kw">def start_engine("kw">self): "kw">pass

v = Vehicle()    # ماذا يحدث؟
```

أ هيشغل عادي وهيعمل object فاضي

ب هيطبع تحذير بس هيكمل

ج هيحصل TypeError لأن مينفعش تعمل instance من abstract class

د هيحصل ImportError

الكود ده فيه bug — إيه المشكلة والتي هيحصل لما بنشغله؟

```
"kw">class Employee:
    "kw">def __init__("kw">self, name, salary):
        "kw">self.name      = name
        "kw">self.__salary  = salary

    "kw">def set_salary("kw">self, amount):
        "kw">if amount > 0:
            "kw">self.__salary = amount

emp = Employee("Ali", 5000)
emp.set_salary(-1000)
"kw">print(emp.__salary)
```

أ هيطبع -1000 لأن الـ setter مش بيشتغل

ب هيطبع 5000 لأن الـ setter رفض السالب، ورفضو AttributeError لأن __salary مش متاحة من بره

ج هيطبع 5000 بدون أي error

د هيطبع 0

تتبع الكود ده بالضبط — إيه الـ output الكامل؟

```
"kw">class Counter:
    count = 0
    "kw">def __init__("kw">self):
        Counter.count += 1
        "kw">self.id = Counter.count

a = Counter()
b = Counter()
c = Counter()
"kw">print(Counter.count)
"kw">print(a.id, b.id, c.id)
```

أ 3 2 1 3

ب 1 1 1 1

ج 3 3 3 3

د 3 2 1 0

Python بتستخدم MRO (Method Resolution Order) عشان تحدد أنهي method تنفذ. الكود ده هيطلع إيه؟

```
"kw">class A:
    "kw">def greet("kw">self): "kw">return "A"

"kw">class B(A):
    "kw">def greet("kw">self): "kw">return "B"

"kw">class C(A):
    "kw">def greet("kw">self): "kw">return "C"

"kw">class D(B, C):
    "kw">pass

d = D()
"kw">print(d.greet())
```

A ☐

B ☐

C ☐

Error: ambiguous inheritance ☐